

Вступ до JavaScript

(частина 1)

План

Огляд JavaScript

Додавання JavaScript на веб-сторінку

Консоль розробника

Синтаксис

Змінні

Коментарі

Типи даних

Взаємодія з користувачем

Огляд JavaScript

JavaScript є найпопулярнішою і широко використовуваною мовою сценаріїв на стороні клієнта.

JavaScript призначена для додавання інтерактивності та динамічних ефектів до веб-сторінок шляхом маніпулювання контентом, який повертається з веб-сервера.

JS підтримує об'єктно-орієнтований, імперативний та функціональний стилі програмування.

Повна інтеграція HTML/CSS.

Підтримується всіма браузерами і вбудована в них за замовчуванням.

Оскільки JavaScript інтегрована у середовище браузера, вона має доступ до інформації про браузер, який зараз використовує користувач.

JavaScript підключається безпосередньо до HTML і запускається негайно, без будь-яких попередніх процесів.

Огляд JavaScript. Історія



JavaScript була створена у 1995 році Брендоном Айком (Brendan Eich) з Netscape Communications як мова сценаріїв у Netscape Navigator. **Перша назва** JavaScript була "**Mocha**". В той самий рік, під час тестування бета-версій Netscape Navigator 2.0, **назву було змінено на "LiveScript"**. Зміна назви відбулася у той час, коли компанія Netscape уклала стратегічне партнерство з компанією Sun Microsystems (яка розробляла мову програмування Java). Основна ідея полягала в тому, що Java повинна була призначатися для великих розробників та професійних програмістів, тоді як **Mocha повинна була використовуватися для невеликих скриптових завдань**.

На формування синтаксису вплинули C і Java, і оскільки Java була дуже популярна в той час, LiveScript був перейменований в JavaScript в 1995 році.

Спочатку JavaScript мала дуже мало функцій. **Її метою було просто додати трохи інтерактивності веб-сторінці**. Наприклад, обробляти натискання кнопки на веб-сторінці, виконувати будь-які інші дії, які в першу чергу стосуються елементів керування.

Однак розвиток веб-середовища, **поява HTML5 і технології Node.js** відкрили набагато більші горизонти для JavaScript. Зараз JavaScript продовжує використовуватися для створення веб-сайтів, але тепер вона має набагато більше можливостей

Огляд JavaScript. ECMAScript

ECMAScript
≠
JavaScript ?

ECMAScript (European Computer Manufacturers Association Script) - це стандарт, який визначає специфікацію скриптової мови програмування. JavaScript є найвідомішою реалізацією цього стандарту, однак є і інші мови, які відповідають даному стандарту, такі як JavaScript, JScript (Microsoft), ActionScript (Adobe Flash) та інші.

Стандарт ECMAScript розробляється та підтримується організацією Ecma International. Офіційні **документи стандарту описують** граматику, типи даних, об'єкти, функції та інші **конструкції мови**.

Стандарт **ECMAScript оновлюється** ітеративно, з новими версіями, що виходять **щороку** або щода кількох років. Кожна нова версія стандарту додає нові функції, поліпшення та коригує недоліки.

ECMAScript і **JavaScript** грають ключову роль у веб-розробці, забезпечуючи можливість взаємодії з користувачем на веб-сторінках та створення багатофункціональних веб-додатків.

ECMAScript 5 - видання стандарту, затверджене в **2009** році

ECMAScript 6 (ES6) – видання стандарту, затверджене **в 2015 році**, яке також називають ECMAScript 2015 (ES2015). Майбутні версії мови називаються відповідно до шаблону ES [YYYY]. Останніми стандартами ECMAScript є ES2015 (або ES6), ES2016, ES2017, ES2018, ES2019, ES2020, ES2021, ES2022, ES2023 ...

Огляд JavaScript. Область застосування

JavaScript має широку область застосування, особливо в контексті веб-розробки:

Клієнтська сторона. JavaScript використовується для взаємодії з користувачем на веб-сторінках. Він дозволяє динамічно змінювати вміст сторінки, обробляти події, валідувати дані форм та виконувати інші завдання, що роблять веб-сторінки більш динамічними та інтерактивними.

Серверна сторона (Node.js). JavaScript також використовується для розробки серверної частини додатків за допомогою платформи Node.js. Це дозволяє використовувати одну мову програмування (JavaScript) і на клієнтській, і на серверній стороні.

Мобільна розробка. Розробники використовують фреймворки, такі як **React Native** або **Ionic**, щоб створювати мобільні додатки за допомогою JavaScript. Це дозволяє використовувати спільний код для розробки додатків для різних платформ (iOS та Android).

Ігрова розробка. JavaScript використовується для створення веб-ігор, включаючи браузерні ігри та ігри на платформах як Steam.

Інтернет в реальному часі (Real-time Web). За допомогою технологій, таких як **WebSockets** або **AJAX**, JavaScript може забезпечити можливість обміну даними в реальному часі між клієнтом та сервером, що корисно для чатів, стрімінгу та інших сценаріїв.

Огляд JavaScript. Можливості

Змінювати вміст HTML-елементів, додавати нові теги, змінювати стилі

Додавати різні ефекти анімації

Реагувати на події - обробляти переміщення вказівника миші, натискання клавіш з клавіатури

Здійснювати перевірку введення даних у поля форми до відправки на сервер, що знімає додаткове навантаження з сервера

Створювати та зчитувати cookie, витягувати дані про комп'ютер відвідувача

Визначати параметри браузера

Відсилати запити на сервер, завантажувати дані із сервера без перезавантаження сторінки (AJAX)

Робота з локальним сховищем даних

Огляд JavaScript. Обмеження

JavaScript не може закривати вікна та вкладки, які не були відкриті за його допомогою

Не може захистити вихідний код сторінки та заборонити копіювання тексту або зображень зі сторінки.

Не може здійснювати кросдоменні запити, отримувати доступ до веб-сторінок, які розташовані на іншому домені. Навіть якщо сторінки з різних доменів відображаються одночасно в різних вкладках браузера, код JavaScript, що належить одному домену, не матиме доступу до інформації про веб-сторінку з іншого домену. Це гарантує безпеку приватної інформації, яка може бути відома власнику домену, сторінка якого відкрита у сусідній вкладці

Не має доступу до файлів, розташованих на комп'ютері користувача, та доступу за межі самої веб-сторінки, винятками є файли cookie (невеликі текстові файли, які JavaScript може записувати та зчитувати) та вибір файлів самим користувачем (`<input type="file">`)

В цілому, можна сказати, що JS розроблена таким чином, щоб утруднити виконання шкідливого коду.

Огляд JavaScript. Виконання коду

JavaScript можна виконувати не тільки в браузері, а й скрізь, для цього потрібна лише спеціальна програма - **інтерпретатор**. Процес виконання коду (сценарію) називається «**інтерпретацією**».

Компіляція - це коли вихідний код програми за допомогою спеціального інструменту (іншої програми) під назвою "**компілятор**" перетворюється на іншу мову, зазвичай - машинний код. Цей машинний код потім поширюється і запускається. Вихідний код програми залишається у розробника.

Інтерпретація – це коли вихідний код програми отримує інший інструмент, який називається «інтерпретатор», і виконує його «як є». Таким чином вихідний код (скрипт) поширюється. Цей підхід використовується в браузерах JavaScript.

Сучасні інтерпретатори перетворюють код JavaScript у машинний код, оптимізують, а вже потім виконують. І навіть під час виконання вони намагаються оптимізувати код. Тому JavaScript працює дуже швидко. Інтерпретатор JavaScript вбудований у всі основні браузери.

Огляд JavaScript. Середовища розробки

"Easy" code editors



- Visual Studio Code



- Sublime Text



- Atom



- Brackets

IDE (Integrated Development Environment)



- WebStorm



- Aptana



- Netbeans

Додавання Java Script до web-сторінки

Зазвичай є **три способи додати JavaScript** до веб-сторінки:

1. Розміщення коду JavaScript безпосередньо **всередині тегу HTML** за допомогою спеціальних атрибутів тегу, таких як *onclick*, *onmouseover*, *onkeypress*, *onload* тощо.
2. Вбудовування коду JavaScript між парою тегів **<script>** і **</script>**.
3. Створення **зовнішнього файлу JavaScript** з розширенням .js, а потім завантаження його на сторінку за допомогою атрибута *src* тегу **<script>**.

Додавання Java Script до web-сторінки всередині тегу HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Including JavaScript</title>
</head>
<body>
  <button onclick="alert('Hello in JavaScript world!')">Click Me</button>
</body>
</html>
```



Слід уникати розміщення великої кількості коду JavaScript в коді HTML, оскільки це загрозаджує HTML та ускладнює підтримку коду JavaScript.

Додавання Java Script до web-сторінки.

Embedding between <script> and </script> tag

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Embedding JavaScript</title>
</head>
<body>
  <script>
    document.write("Hello in JavaScript world!"); // Prints: Hello in JavaScript world!
  </script>
</body>
</html>
```

Note: The type attribute for <script> tag (i.e. <script type="text/javascript">) is no longer required since HTML5.

Додавання Java Script до web-сторінки.

External JavaScript file

Якщо коду JavaScript багато, він виводиться в окремий файл, який з'єднується в HTML за допомогою елемента script з атрибутом src:

```
<script src="./path/to/script.js"></script>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Including External JavaScript File</title>
</head>
<body>
  <h1>Some title</h1>
  <script src="/javascript/hello.js"></script>
</body>
</html>
```

External file hello.js:

```
document.write("Hello in JavaScript world!");
```

Додавання Java Script до web-сторінки.

External JavaScript file

Для підключення декількох скриптів використовується кілька елементів **script** :

```
<script src="./script1.js">alert()</script>
```

```
<script src="./script2.js"></script>
```

Як правило, в HTML-кодi пишуться лише найпростіші скрипти, а складні виносяться в окремий файл. Завдяки цьому один і той же скрипт, наприклад, меню або бібліотеку функцій, можна використовувати на різних сторінках. Браузер завантажує його лише вперше і в майбутньому, якщо сервер налаштовано правильно, він забере його зі свого кешу.



Зауважте, що якщо вказано атрибут **src**, то **вміст тегу ігнорується**. В одному елементі **script** неможливо одночасно підключити зовнішній скрипт і вказати код



Ви повинні намагатися завжди тримати вміст і структуру вашої веб-сторінки (HTML) окремо від форматування (CSS) і поведінки (JavaScript).

Тег <noscript>

Тег <noscript> використовується в HTML для надання запасного вмісту, який буде відображено, якщо в браузері вимкнено або не підтримується виконання JavaScript.

<noscript>

<!-- Альтернативний вміст для користувачів без JavaScript -->

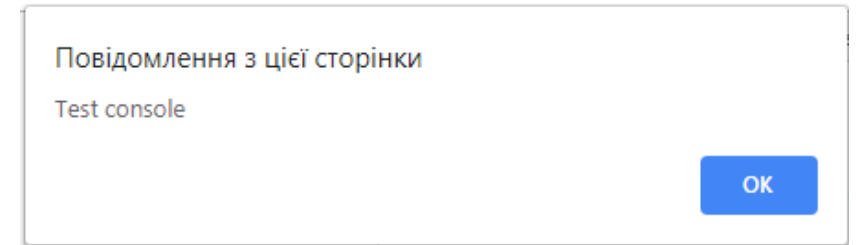
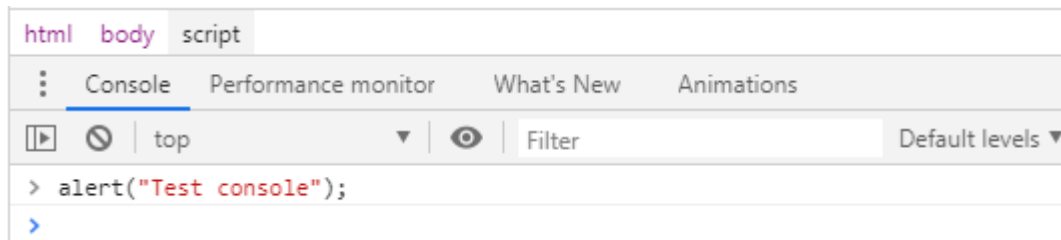
</noscript>

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>No script tag</title>
</head>
<body>
  <h1>Example Page</h1>
  <p>This page uses JavaScript to enhance user experience.</p>
  <noscript>
    <p>Please enable JavaScript to view the content.</p>
  </noscript>
</body>
</html>
```


Консоль розробника

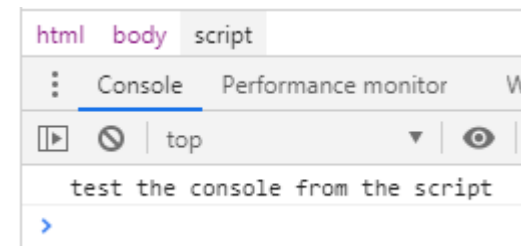
Незамінним інструментом при роботі з JavaScript є **консоль** браузера, яка дозволяє відналаджувати (debug) програму. Багато сучасних браузерів мають подібну консоль. Найпростіше **відкрити** консоль за допомогою клавіші **F12** (також можна скористатися меню браузера).

1) Ви можете **безпосередньо вводити** вирази JavaScript у консоль браузера, і вони виконуються:



2) Щоб відобразити інформацію в консолі браузера, скористайтеся функцією **console.log()**:

```
<body>
  <h1>Some title</h1>
  <script>
    console.log("test the console from the script");
  </script>
</body>
```



Синтаксис

Код JavaScript складається з інструкцій (команд), кожна з яких закінчується крапкою з комою:

```
alert("Some text 1"); alert(" Some text 2");
```

Однак сучасні браузері цілком можуть розрізняти окремі інструкції, якщо вони просто розташовані в окремих рядках без крапки з комою:

```
alert(" Some text 1")  
alert(" Some text 2")
```

Але щоб покращити читабельність коду та зменшити кількість можливих помилок, рекомендується писати кожен рядок JavaScript в окремому рядку і закінчувати її крапкою з комою:

```
alert("Some text 1"); // best practice!  
alert("Some text 2"); // best practice!
```

Літерали, змінні. Ініціалізація

Синтаксис JavaScript визначає два типи значень: **фіксовані значення і змінні значення**.

Фіксовані значення називаються **літералами** (52, 8.4, «Petro Petrenko»). Значення змінних називаються змінними.

Змінні є основними для всіх мов програмування. Змінні використовуються для зберігання даних, як-от рядок тексту, чисел тощо. Дані або значення, що зберігаються у змінних, можна встановлювати, оновлювати та отримувати, коли це необхідно. Загалом, змінні є символічними іменами значень.

Змінна складається з імені та виділеної області пам'яті, яка їй відповідає.

Щоб створити (оголосити, визначити) змінну в JavaScript використовують одне з ключових слів: **var**, **let** чи **const**:

var data; // old syntax

let data; // new syntax

Після створення можна записати дані до змінної, використовуючи **знак рівності =** (присвоєння):

let data;

data = 200; // writing a number to a variable

Процес запису **початкового** значення називається **ініціалізацією**.

Регістрозалежність імен змінних

JavaScript є мовою, яка враховує регістр при оголошенні змінних. При оголошенні змінних їх ідентифікатори є регістронезалежними. Це означає, що використання змінної залежить від того, як вона була оголошена.

```
const myVariable = "Hello";  
const MyVariable = "World";  
console.log(myVariable); // Виведе "Hello"  
console.log(MyVariable); // Виведе "World "
```

У цьому прикладі `myVariable` та `MyVariable` вважаються різними змінними через різний регістр літер.

Треба бути обережним при роботі з регістрозалежними ідентифікаторами, оскільки це може призвести до плутанини та помилок в коді. Зазвичай рекомендується використовувати консистентний стиль іменування, щоб уникнути таких проблем. Багато розробників використовують `camelCase` для іменування змінних (наприклад, `myVariable`), що дозволяє уникнути плутанини через регістр.

Змінні. Доступ та зміна

Записані дані будуть збережені у відповідній області пам'яті та згодом доступні при доступі за **ім'ям**:

```
let data;  
data = "Some text";  
console.log(data); // Display the contents of a variable
```

Для більш стислого запису можна **поєднати** створення та ініціалізацію змінної:

```
let data = "Some text";
```

Значення змінної **можна змінювати** скільки завгодно разів:

```
let data = "Some text";  
data = "Another text"; // Change the value of a variable  
console.log(data); // "Another text"
```

При зміні значення старе значення **видаляється**

Копіювання змінних

Змінні в JavaScript можуть зберігати не тільки рядки, а й інші дані, наприклад числа.

Створимо дві змінні, помістимо в одну - рядок, а в іншу - число. Змінній неважливо від того, що зберігати:

```
let data = 25;
```

```
let note = "Text";
```

Значення **можна скопіювати** з однієї змінної в іншу:

```
let data = 25;
```

```
let note = "Text";
```

```
note = data;
```

Значення з data **перезаписує** поточне значення в note

Після цього присвоєння змінні data та note будуть містити те саме значення 25

Константи

Константа - це змінна, яка ніколи не змінюється. Щоб оголосити константу, використовується ключове слово **const**:

```
const E = 2,71828;
```

```
const PI = 3,14;
```

Константним змінним JavaScript необхідно призначити значення, коли вони оголошуються:

```
const PI;  
PI = 3.14; // bad usage
```

```
const PI = 3.14; // good usage
```

Константи використовуються замість рядків і чисел для того, щоб зробити програму більш зрозумілою та уникнути помилок.

Правила присвоєння імен змінних JS (Naming Conventions)

Вибір правильного імені змінної є однією з найважливіших і складних речей у програмуванні. Справа в тому, що більшість часу ми витрачаємо не на початкове написання коду, а на його розробку.

Існують такі правила для іменування змінних JavaScript:

- 1) Ім'я змінної **може** починатися з літери, символу підкреслення (`_`) або знака долара (`$`) (`_x`, `$result`)
- 2) Ім'я змінної **НЕ може** починатися з числа (`1x`)
- 3) Ім'я змінної може містити **лише** буквено-цифрові символи (A-z, 0-9) і підкреслення (`let my-data`)
- 4) Ім'я змінної **не може містити пробіли** (`const my data`)
- 5) Змінні з кількох слів записуються у стилі camelCase (`let myNewVariable`)
- 6) Не варто використовувати транслітерацію - тільки англійська (`let imya` ---> `let name`)
- 7) Ім'я змінної має **максимально чітко відповідати** даним, що в ній зберігаються
- 8) Ім'я змінної не може бути **ключовим словом** JavaScript або **зарезервованим словом** JavaScript
- 9) Змінні JavaScript **чутливі до регістру** (`sum`, `Sum` та `SUM` – різні змінні)

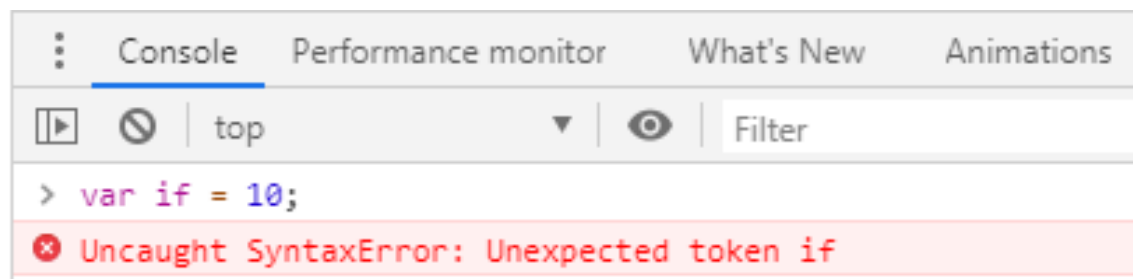
JavaScript variable name validator: <https://mothereff.in/js-variables>

Зарезервовані слова

Існує список **зарезервованих слів**, які не можна використовувати при іменуванні змінних, оскільки вони використовуються самою мовою:

abstract, boolean, break, byte, case, catch, char, class, const, continue, debugger, default, delete, do, double, else, enum, export, extends, false, final, finally, float, for, function, goto, if, implements, import, in, instanceof, int, interface, long, native, new, null, package, private, protected, public, return, short, static, super, switch, synchronized, this, throw, throws, transient, true, try, typeof, var, volatile, void, while, with

Наступний приклад призведе до синтаксичної помилки:



Коментарі

З часом програма стає великою і складною. Тому необхідно додавати коментарі, які пояснюють, що відбувається і чому. **Коментарі** можуть бути **розташовані** в будь-якому місці програми і **жодним чином не впливати** на її виконання. Інтерпретатор JavaScript просто ігнорує їх.

Однорядкові коментарі починаються з подвійної косої риски **//**. Текст вважається коментарем до кінця рядка:

```
// Message output  
alert("Message");  
alert("Another message"); // Message output
```

Багаторядкові коментарі починаються зі зірочки **/*** і закінчуються зірочкою ***/**:

```
/* Messages  
output */  
alert("Message");  
alert("Another message");
```

Типи даних

Data Type	Description
String	represents sequence of characters
Number, BigInt	represents numeric values
Boolean	represents boolean value either false or true
Undefined	represents undefined value
Null	represents null
Symbol	represents unique identifier
Object	represents instance through which we can access members



primitive data types



non-primitive data type

Типи даних. Number

Числа (тип **Number**) в JavaScript можуть мати дві форми - цілі і дробові числа (числа з плаваючою комою). Ми можемо використовувати як додатні, так і від'ємні числа.

```
const x = 45;
```

```
const y = -23.897;
```

Існують спеціальні числові значення **Infinity/-Infinity** і **NaN** (Not-a-Number, помилка обчислення). Вони також належать до типу Number.

Наприклад, нескінченність *infinity* отримується шляхом ділення на нуль:

```
console.log(100 / 0); // Infinity
```

А *-Infinity* можна отримати, поділивши від'ємне число на нуль:

```
console.log(-100 / 0); // -Infinity
```

Помилка обчислення NaN буде результатом неправильної математичної операції:

```
console.log("some string" * 5); // NaN
```

Типи даних. String. Boolean

Тип **string** представляє рядки, тобто дані, узяті в лапки:

```
const str = "Hello my friend!";
```

```
const str2 = 'Single quotes will do too';
```

У JavaScript **одинарні** та **подвійні лапки** рівносильні. Можна використовувати або ті, або інші. Варто дотримуватися **одного стилю лапок** по всьому проекту

Boolean – логічний тип даних. Він має лише два значення - **true** і **false**.

Зазвичай цей тип використовується для зберігання значень так/ні:

```
let isValid = true;
```

```
isValid = false;
```

Типи даних. null. undefined

null – окремий тип, що складається з одного нульового значення. Призначення змінній значення null означає, що ця змінна має деяке невизначене значення (не число, не рядок, не логічне значення), але все ж вона має значення:

```
let age = 15;
```

```
age = null; // value deletion
```

undefined – спеціальне значення, яке, як і **null**, утворює власний тип. Це означає, що значення змінній не присвоєно.

Якщо змінна **оголошена**, але **в неї нічого не записано**, то її значення просто **невизначене**:

```
let u;
```

```
console.log(u); // undefined
```

Змінній можна призначити **undefined** і явно:

```
let x = 123;
```

```
x = undefined;
```



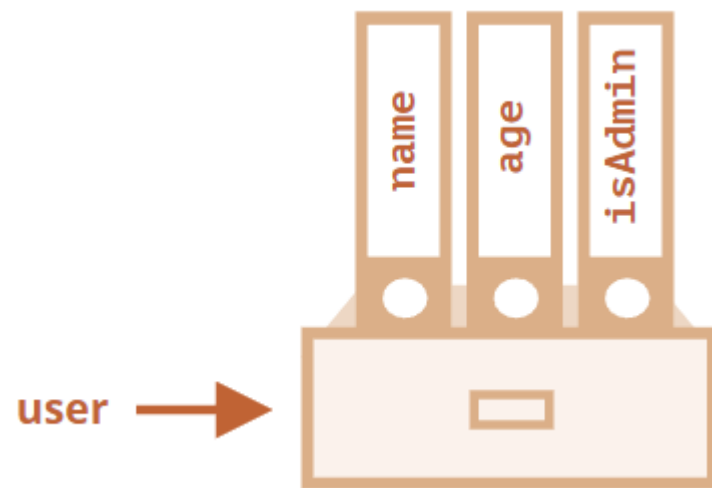
У явній формі **undefined** зазвичай не призначається, оскільки це суперечить його сенсу. **null** використовується для запису в змінну **null**

Типи даних. Object

В JavaScript, тип даних **object** є одним з базових типів, і він використовується для представлення об'єктів — наборів **ключ-значення**, де кожне значення асоційоване з унікальним ключем. Об'єкти використовуються для представлення **складних структур** даних, а також для реалізації різних **паттернів програмування**.

Найпростіше оголошення об'єкта здійснюється фігурними дужками:

```
const newObj = {};  
const user = {           // an object  
  name: "John",          // by key "name" store value "John"  
  age: 30,                // by key "age" store value 30  
  isAdmin: true           // by key "isAdmin" store value true  
};  
user.name = "Kira";       // set key "name" to value "Kira"  
user[age] = 28;           // set key "age" to value 28
```



Існує можливість динамічно додавати, змінювати або видаляти ключі та їхні значення у вже існуючих об'єктах.

Типи даних. Динамічна (слабка) типізація

JavaScript — це мова з динамічною (слабкою) типізацією. Це означає, що змінні можуть динамічно змінювати тип під час виконання:

```
let x; // type undefined  
console.log(x); // undefined
```

```
x = 45; // type number  
console.log(x); // 45
```

```
x = "45"; // type string  
console.log(x); // "45"
```

Хоча в другому і третьому випадку буде виведено 45, але в другому випадку змінна `x` представлятиме число, а в третьому випадку рядок.

Причому змінна `x` може змінювати свій тип даних

Типи даних. `typeof`

`typeof x = typeof (x)`

Використовуючи ключове слово `typeof`, ви можете отримати тип змінної. Результатом **`typeof`** є рядок, що містить тип.

```
let name = "Tom";  
console.log(typeof name); // "string"
```

```
let income = 45.8;  
console.log(typeof income); // "number"
```

```
let isEnabled = true;  
console.log(typeof isEnabled); // "boolean"
```

```
let undefVariable;  
console.log(typeof undefVariable); // "undefined"
```

```
let nullVariable = null;  
console.log(typeof nullVariable); // "object"
```

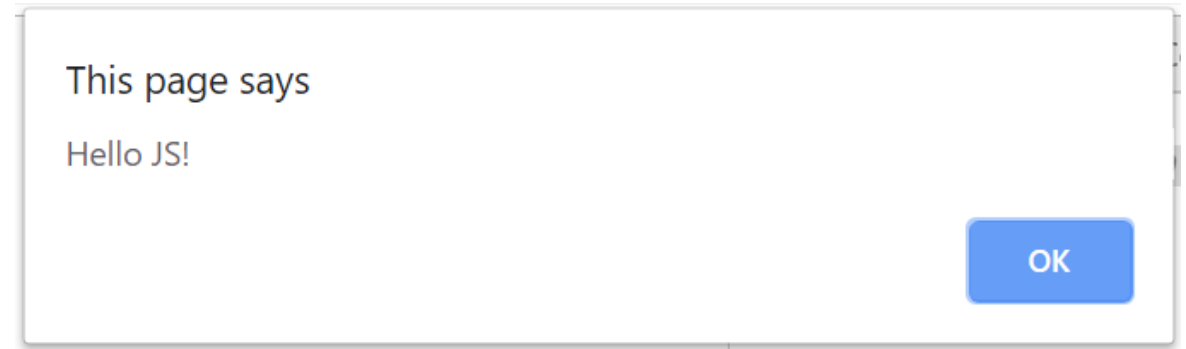


Для `null`, `typeof` повертає об'єкт, це офіційно визнана помилка в мові, яка збережена для сумісності

Взаємодія з користувачем. `alert()`

Функція `alert()` відображає вікно повідомлення та призупиняє сценарій, доки користувач не натисне OK:

```
alert("Hello JS!");
```



Вікно повідомлення, яке відображається, є **модальним вікном**. Слово модальне означає, що **користувач не може взаємодіяти зі сторінкою**, натискати інші кнопки тощо, **поки не закриє вікно**. У цьому випадку до тих пір, поки не клацне OK

Взаємодія з користувачем. `prompt()`

Функція `prompt` приймає два аргументи:

```
result = prompt(title, default);
```

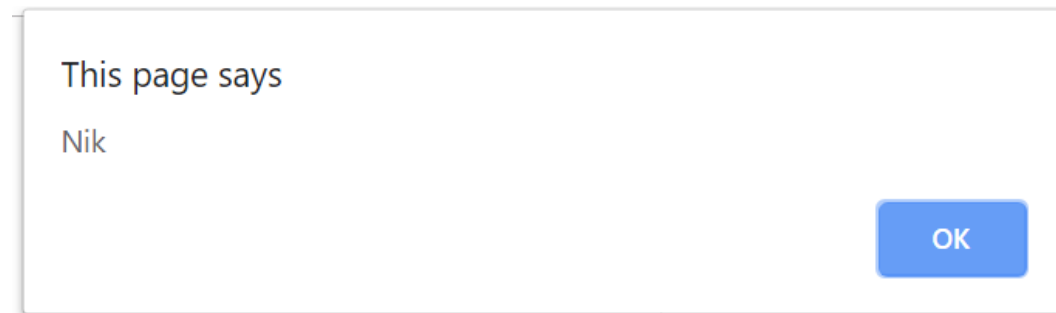
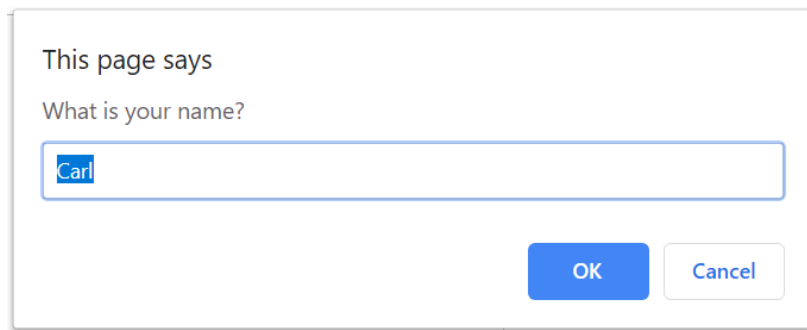
Вона виводить модальне вікно із текстом `title`, полем введення тексту, заповненим рядком за замовчуванням, і кнопками OK/CANCEL.

Користувач повинен або **ввести щось** і **натиснути OK**, або **скасувати** введення, натиснувши СКАСУВАТИ або натиснувши Esc на клавіатурі.

Виклик `prompt` повертає те, що ввів відвідувач - **рядок** або спеціальне нульове (**null**) значення, якщо введення скасовано.

```
let name = prompt("What is your name?", "Carl");
```

```
alert(name);
```



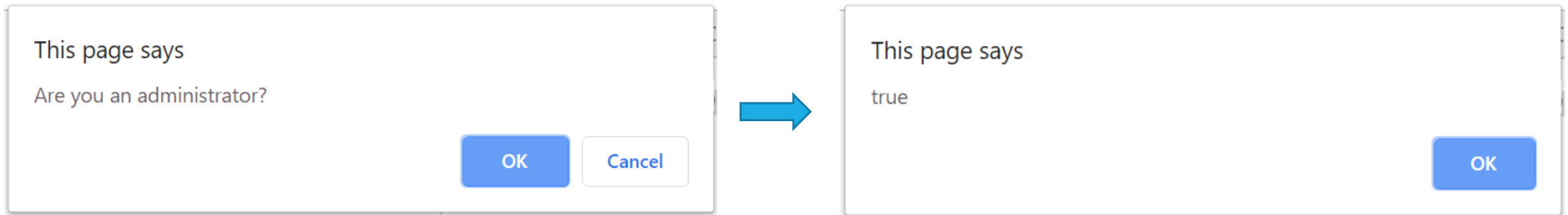
Взаємодія з користувачем. **confirm()**

Синтаксис:

```
result = confirm(question);
```

confirm() виводить вікно запитання з двома кнопками: OK і CANCEL. **Результат** буде **true**, коли ви натиснете кнопку OK, і **false**, якщо СКАСУВАТИ (Esc).

```
let isAdmin = confirm("Are you an administrator?");  
alert( isAdmin );
```



Підсумки

Познайомилися з JavaScript

Навчилися додавати код JavaScript до веб-сторінки

Ознайомилися з консоллю розробника

Вивчили синтаксис мова

Навчилися використовувати змінні

Навчилися писати коментарі в коді JS

Вивчили типи даних JS. Навчилися визначати тип даних для змінної.

Навчилися налаштовувати за допомогою JS взаємодію з користувачем

Дякую за увагу...
